

C# / .NET Learning Roadmap

Topic Notes — Section 8: Memory Management (Lightweight)

Topics: 8.1 through 8.2

Date: 29 June 2026

Section 8 — Memory Management (Lightweight)

This section covers both roadmap topics: 8.1 (Stack vs heap — concept only) and 8.2 (IDisposable and the using statement/declaration).

Note on scope: Your roadmap marks this section as Lightweight — meaning conceptual understanding, not deep internals. The focus is on what every .NET developer needs to know daily, not GC internals or finalizer patterns (those are Tier 2).

8.1. Stack vs Heap (Concept Only)

Builds on: 1.4 (value vs reference types — the stack/heap distinction explains why they behave differently), 1.16 (structs vs classes — structs are value types, classes are reference types), 2.1 (classes — class instances live on the heap).

What are the stack and heap?

When your program runs, the .NET runtime manages two regions of memory where data lives:

The Stack — fast, ordered, automatic:

- Stores local variables, method parameters, and return addresses
- Works like a stack of plates — items are added and removed from the top (LIFO)
- Memory is allocated and freed automatically as methods are called and return
- Fixed size (typically 1MB per thread)
- No garbage collector involvement — instant cleanup

The Heap — flexible, unordered, managed:

- Stores objects (class instances) and their data
- Memory is allocated on demand, can grow large
- The Garbage Collector (GC) periodically scans and frees objects no longer referenced
- Slower allocation than stack; GC collection has a cost

[illegible]

Value types go on the stack, reference types go on the heap

Value types (int, decimal, bool, struct, enum) — stored directly where they're declared:

```
void ProcessOrder(int orderId, decimal amount)
{
    // orderId and amount live on the stack:
    int tax = (int)(amount * 0.18m);    // tax also on the stack
    // When this method returns, all three are instantly freed
}
```

Reference types (class, string, arrays, delegates) — the variable on the stack holds a reference (pointer) to the actual object on the heap:

```
void CreateOrder()
{
    // 'order' variable lives on the stack -- holds a reference
    // The actual Order object lives on the heap
    Order order = new Order { Id = 1, Amount = 100m };

    // When 'order' goes out of scope, the reference is gone
    // The Order object on the heap is eventually collected by GC
}
```

Why the distinction matters practically

1. Value types are copied, reference types are shared:

```
// Value type -- copy is made:
int a = 5;
int b = a;    // b is a copy
b = 10;
Console.WriteLine(a);    // 5 -- a is unchanged

// Reference type -- reference is shared:
var orderA = new Order { Amount = 100m };
var orderB = orderA;      // orderB points to the SAME object
orderB.Amount = 999m;
Console.WriteLine(orderA.Amount);    // 999 -- same object!
```

2. Method parameters — value vs reference semantics:

```
// Value type parameter -- method gets a copy:
void Double(int x) { x *= 2; }
int n = 5;
Double(n);
Console.WriteLine(n);    // 5 -- method worked on a copy

// Reference type parameter -- method gets the same reference:
void SetAmount(Order order) { order.Amount = 999m; }
var o = new Order { Amount = 100m };
SetAmount(o);
Console.WriteLine(o.Amount);    // 999 -- same object modified
```

3. Null — only reference types can be null by default:

```
int n = null;    // compile error -- int is a value type
Order o = null;    // valid -- Order is a reference type

// Nullable value types (? syntax):
int? n = null;    // valid -- Nullable<int> wraps the value
```

The Garbage Collector — what it does

The GC is the .NET runtime component that automatically frees heap memory. You don't free memory manually in C# — the GC handles it.

```
void CreateAndForget()
{
```

```

    var order = new Order { Id = 1 };    // Order allocated on heap
}    // 'order' variable goes out of scope
    // The Order object has no live references -- eligible for GC
    // GC will collect it at some future point (not immediately)

```

The GC organises objects into three generations:

- Gen 0 — recently allocated, short-lived objects — collected most frequently
- Gen 1 — objects that survived Gen 0 — collected less frequently
- Gen 2 — long-lived objects — collected rarely

Most objects in a well-written app are short-lived (Gen 0) — allocated for a request and freed when the request completes. The GC is optimised for this pattern.

What you need to know vs what you don't

Need to know (daily relevance):

- Value types are copied, reference types share the same object
- Null only applies to reference types by default (use ? for nullable value types)
- Objects on the heap are collected by GC — you don't free them manually
- Large objects or object graphs that hold references prevent GC collection (memory leaks)

Do not need to know (Tier 2 -- deep dive only when needed):

- GC internals — LOH, SOH, pinned objects
- Finalizers and the finalization queue
- GC.Collect(), GC.SuppressFinalize()
- Span<T>, Memory<T>, stackalloc (stack allocation for arrays)
- Boxing and unboxing performance details

Key Takeaways for 8.1

- Stack — fast, automatic, local variables and value types, freed when method returns
- Heap — flexible, managed by GC, class instances and reference types
- Value types (struct, int, decimal) are copied on assignment
- Reference types (class) share the same object -- changes visible through any reference
- GC automatically frees unreachable heap objects -- no manual memory management
- The practical implication you need daily: copying vs sharing, and what can be null

Memory aid: Stack is a notepad on your desk — quick to write on, wiped clean when you're done. Heap is a filing cabinet — stores things longer, needs a cleaner (GC) to come periodically and throw out old files.

8.2. IDisposable and the using Statement/Declaration

Builds on: 8.1 (stack vs heap — IDisposable is about releasing heap resources), 2.9 (interfaces — IDisposable is an interface), 7.3 (async/await — IAsyncDisposable and await using extend this to async).

What is IDisposable and why does it exist?

The GC handles memory automatically. But some resources are not memory — they are OS-level handles, network connections, file handles, database connections. The GC doesn't know how to release these, and their cleanup timing is unpredictable.

Unmanaged resources -- GC cannot clean these up automatically:

- File handles (FileStream)
- Database connections (SqlConnection, DbContext)
- Network sockets (HttpClient, TcpClient)
- Cryptographic handles
- Window handles (GDI objects)

If you open a file and don't close it, the file handle stays open until the process exits. 1000 leaked database connections will exhaust the connection pool. IDisposable is the contract for releasing unmanaged resources deterministically — at a known point in time.

```
public interface IDisposable
{
    void Dispose();
}
```

The using statement — automatic Dispose

The using statement guarantees Dispose() is called when the block exits — even if an exception is thrown:

```
// Without using -- easy to forget Dispose:
var stream = new FileStream("data.txt", FileMode.Open);
var data = stream.ReadToEnd();
stream.Dispose(); // what if ReadToEnd throws? Dispose never called!

// With using -- Dispose is always called:
using (var stream = new FileStream("data.txt", FileMode.Open))
{
    var data = stream.ReadToEnd();
} // Dispose called here automatically, even if exception thrown
```

The compiler transforms using into a try/finally:

```
// What the compiler generates from using:
var stream = new FileStream("data.txt", FileMode.Open);
try
{
    var data = stream.ReadToEnd();
}
finally
{
    stream?.Dispose(); // always called
}
```

using declaration — C# 8+ simplified syntax

C# 8 introduced the using declaration — no braces needed. The variable is disposed when it goes out of scope:

```
// Traditional using statement with braces:
using (var connection = new SqlConnection(connectionString))
```

```

{
    connection.Open();
    // use connection
} // disposed here

// C# 8+ using declaration -- no braces:
using var connection = new SqlConnection(connectionString);
connection.Open();
// use connection
// disposed at end of method/block -- same guarantee

// Multiple disposables -- C# 8+ is much cleaner:
using var conn = new SqlConnection(connStr);
using var cmd = conn.CreateCommand();
// both disposed at end of method in reverse order

```

What to use with using

```

// Database:
using var connection = new SqlConnection(connectionString);
using var context = new ApplicationDbContext(options); // when created manually

// File I/O:
using var stream = new FileStream("file.txt", FileMode.Open);
using var reader = new StreamReader(stream);
using var writer = new StreamWriter("output.txt");

// HTTP response:
using var response = await _httpClient.GetAsync(url, ct);
// Note: HttpClient itself should NOT use 'using' -- it is designed to be reused

// Cryptography:
using var aes = Aes.Create();
using var hash = SHA256.Create();

// CancellationTokenSource:
using var cts = new CancellationTokenSource(TimeSpan.FromSeconds(5));

```

IAsyncDisposable and await using

For resources that require async cleanup (closing a network connection gracefully, flushing an async stream), use `IAsyncDisposable` and `await using`:

```

public interface IAsyncDisposable
{
    ValueTask DisposeAsync();
}

// await using -- calls DisposeAsync() instead of Dispose():
await using var connection = await _pool.GetConnectionAsync();
await connection.ExecuteAsync(query);
// DisposeAsync() called here -- returns connection to pool asynchronously

// IAsyncDisposable with await using for async cleanup:
await using var writer = new StreamWriter("output.txt");

```

```
await writer.WriteLineAsync("Hello");
// DisposeAsync flushes buffer asynchronously before closing
```

Implementing IDisposable -- the standard pattern

```
public class OrderExporter : IDisposable
{
    private readonly FileStream _stream;
    private readonly StreamWriter _writer;
    private bool _disposed;

    public OrderExporter(string filePath)
    {
        _stream = new FileStream(filePath, FileMode.Create);
        _writer = new StreamWriter(_stream);
    }

    public async Task WriteOrderAsync(Order order)
    {
        ObjectDisposedException.ThrowIf(_disposed, this);
        await _writer.WriteLineAsync($"{order.Id},{order.Amount},{order.Status}");
    }

    public void Dispose()
    {
        if (_disposed) return;
        _writer.Dispose(); // dispose in reverse creation order
        _stream.Dispose();
        _disposed = true;
    }
}

// Usage:
using var exporter = new OrderExporter("orders.csv");
foreach (var order in orders)
    await exporter.WriteOrderAsync(order);
// Dispose called automatically -- file closed and flushed
```

DI and disposal in ASP.NET Core

In ASP.NET Core, the DI container automatically disposes IDisposable services when their scope ends:

```
// Scoped service implementing IDisposable:
public class OrderService : IOrderService, IDisposable
{
    private readonly ApplicationDbContext _context;

    public OrderService(ApplicationDbContext context) => _context = context;

    public void Dispose() => _context.Dispose();
}

// Registration:
builder.Services.AddScoped<IOrderService, OrderService>();
```

```
// DI creates OrderService per request and disposes it when request ends
// You never call Dispose() manually -- the container does it
```

The rule: If a service is registered with the DI container, don't manually dispose it. The container owns the lifetime and handles disposal. Only use using on objects you create outside of DI.

Common mistakes

```
// WRONG -- disposed too early:
var context = new AppDbContext(options);
var orders = context.Orders.AsQueryable();
context.Dispose(); // disposed!
var list = orders.ToList(); // ObjectDisposedException!

// CORRECT -- dispose after materialising:
using var context = new AppDbContext(options);
var list = await context.Orders.ToListAsync(); // materialised before dispose

// ████████████████████████████████████████████████████████████████████████

// WRONG -- creating new HttpClient per request:
[HttpGet]
public async Task<IActionResult> GetData()
{
    using var client = new HttpClient(); // new per request
    var data = await client.GetStringAsync(url); // socket exhaustion!
    return Ok(data);
}

// CORRECT -- HttpClient is designed to be shared:
// Register in DI and inject:
builder.Services.AddHttpClient<MyService>();

// ████████████████████████████████████████████████████████████████████████

// WRONG -- forgetting to dispose in a loop:
foreach (var filePath in filePaths)
{
    var stream = new FileStream(filePath, FileMode.Open); // never disposed!
    ProcessStream(stream);
}

// CORRECT:
foreach (var filePath in filePaths)
{
    using var stream = new FileStream(filePath, FileMode.Open);
    ProcessStream(stream);
} // stream disposed each iteration
```

Key Takeaways for 8.2

- `IDisposable.Dispose()` releases unmanaged resources deterministically -- file handles, DB connections, sockets

- Always use using (or using var) for any IDisposable -- guarantees Dispose() even if exceptions occur
- using var (C# 8+) -- cleaner syntax, disposed at end of enclosing scope
- IDisposable + await using -- for resources needing async cleanup
- In ASP.NET Core DI -- the container disposes services; don't manually dispose DI-managed objects
- HttpClient should NOT be wrapped in using -- use DI or IHttpClientFactory

Type	Needs using?	Notes
FileStream, StreamReader	Yes	File handles
SqlConnection, DbConnection	Yes	DB connections
DbContext (manual)	Yes	When created manually
DbContext (DI)	No	Container disposes it
HttpClient	No	Reuse via DI/factory
HttpResponseMessage	Yes	From GetAsync
Aes, SHA256, crypto	Yes	Crypto handles
CancellationTokenSource	Yes	Use using var

.NET-specific rules:

- using var is preferred over using (var x = ...) in modern C# -- cleaner, same guarantee
- await using for IDisposable -- never call DisposeAsync().Wait() or .Result
- If a type has Close() alongside Dispose() -- call Dispose() not Close()
- ObjectDisposedException -- thrown when you use an already-disposed object. Fix: restructure so disposal happens after last use

Memory aid: IDisposable is like a library book. The GC is a library that eventually collects unreturned books (memory). But some books have late fees (file locks, DB connections). using is like setting a due-date reminder that returns the book automatically when you're done, regardless of what happened during reading.

Debugging Tips

Symptom	Cause	Fix
ObjectDisposedException	Using a resource after it was disposed	Restructure code so the using scope covers all usage of the resource
File locked after program continues	FileStream not disposed -- file handle still open	Wrap all FileStream access in using var
Database connection pool exhausted	DbContext or SqlConnection not disposed	Ensure every DbContext created manually uses using var
Socket exhaustion -- HttpClient errors	Creating new HttpClient() per request	Use DI with AddHttpClient or IHttpClientFactory
await using not compiling	Type implements IDisposable not IDisposable	Use regular using for IDisposable; only await using for IDisposable

Self-check -- you should now be able to:

8.1:

- Explain the difference between stack and heap and what lives in each
- Explain why value types are copied and reference types share the same object
- Explain what the GC does at a conceptual level

8.2:

- Explain why IDisposable exists and what types of resources need it
- Write using var for any IDisposable and explain what happens if you don't
- Use await using for IAsyncDisposable types
- Explain why HttpClient should not be wrapped in using
- Explain the DI container's role in disposal and when NOT to call Dispose() manually

Section 8 complete. Both roadmap topics (8.1 and 8.2) covered: Stack vs Heap (conceptual) and IDisposable with the using statement and declaration. Say next to begin Section 9: SOLID Principles and Core Design Patterns.